Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO




# LABORATORIO N° 10

# Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

## DESARROLLO DE APLICACIONES EMPRESARIALES:

### Capacidades

1. **Identificar la funcionalidad de los componentes, las props, el estado y los eventos de React dentro de una aplicación empresarial.**
2. **Dividir una interfaz en componentes funcionales reutilizables que reciben sus datos mediante props, incluyendo listas y relaciones anidadas.**
3. **Programar eventos (onClick, onChange y onSubmit) con useState y construir un formulario controlado con validación.**

### I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

### II. Fundamento teórico

- Revise el material de la Semana 10 (Componentes y Eventos I: componentes funcionales, props, estado con useState, eventos y formularios controlados) antes del desarrollo del laboratorio.

### III. Normas empleadas

- No aplica

### IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Node.js (versión LTS, que incluye npm), un editor de código (Visual Studio Code), un navegador web y una cuenta de GitHub para el control de versiones.
- El proyecto de React (carpeta frontend) desarrollado en el laboratorio de la Semana 9, junto con la aplicación de las Semanas 3 a 8 (incluida la API REST de la Semana 8) y, cuando corresponda, la investigación propia de siete entidades.

### V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

### VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (los archivos de src/components y App.jsx) y de la página en el navegador mostrando el resultado de la interacción (antes y después del clic o del envío del formulario).
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

### VII. Procedimiento

En la Semana 9 cada estudiante creó un proyecto de React con Vite y construyó una interfaz completa dentro de App.jsx, con listas, condicionales y una relación anidada, usando datos escritos a mano con la forma de la API de la Semana 8. En esta sesión no se crean datos nuevos: se reorganiza esa misma interfaz en componentes reutilizables que reciben sus datos mediante props y se le agrega interacción con estado y eventos. Todavía no se consume la API (eso se hace en la Semana 12) y no se pasan funciones entre componentes (Semana 11): los eventos y el estado se manejan dentro del componente que los usa.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante retoma el proyecto de la Semana 9 para la aplicación de las Semanas 4 a 8 y lo reorganiza en componentes con estado y eventos. En la segunda parte, deberá hacer lo mismo sobre su investigación propia (las siete entidades registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que un componente es la unidad con la que se organiza una interfaz de React: recibe datos por props, guarda los que cambian en el estado y responde a los eventos del usuario. Lo que en Django era un {% include %} con datos del context, en React es un componente con props. En clase la teoría se demuestra con una aplicación de cafetería; cada estudiante la traslada a su propia aplicación con ayuda de la siguiente tabla de equivalencias.

| **Concepto trabajado en clase** | **Equivalente en su aplicación** |
|---|---|
| Componente de un registro (en clase: ProductoCard, con nombre, precio y stock) | Un componente que muestre un registro de su entidad principal, con un nombre o título, un campo numérico y un campo de estado o cantidad. |
| Botón con estado (en clase: Agregar, con un contador de agregados) | Un botón en el componente del registro que cambie un dato local (contador, reserva, favorito o marca). |
| Componente con relación 1:N (en clase: PedidoCard con DetalleItem) | Un componente para su relación 1:N, 1:1 o modelo intermedio, que use otro componente para cada elemento anidado. |
| Formulario y regla de validación (en clase: nuevo producto, con precio mayor que cero) | Un formulario para agregar un registro a su lista, con una regla de validación tomada de su API de la Semana 8. |

## Navegador (http://localhost:5173/)

## → main.jsx → App.jsx

## → Componentes (src/components)

## → Props (datos hacia los componentes hijos)

## → Estado con useState

## → Eventos: onClick, onChange y onSubmit

## → React vuelve a dibujar el componente

## → Pantalla

Al finalizar el laboratorio, el estudiante deberá ser capaz de organizar una interfaz en componentes funcionales que reciben datos por props, agregar estado con useState, programar eventos de clic y de escritura, construir un formulario controlado con validación y publicar el resultado en GitHub.

**Enunciados**

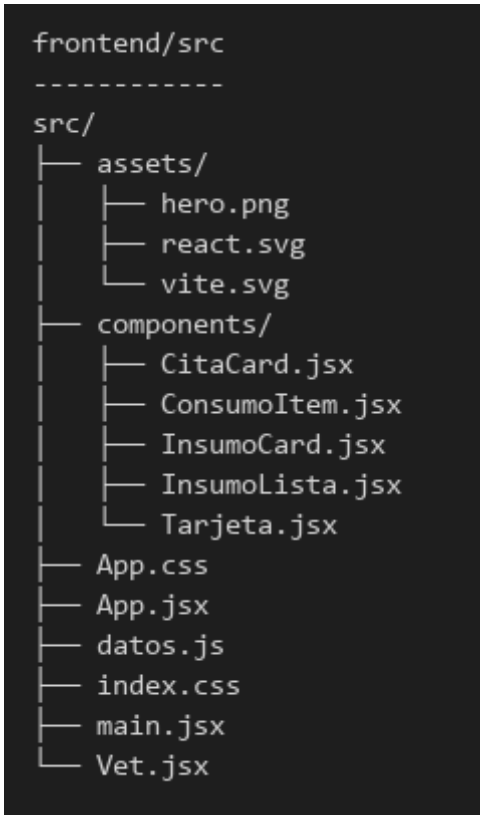
## PARTE 1 — Componentes, estado y eventos para la aplicación de las Semanas 4 a 8

## Ejercicio 1 — Recuperar el proyecto e identificar los componentes

Abra el proyecto de React de la Semana 9 (carpeta frontend) y ejecute npm run dev. Identifique en App.jsx tres fragmentos que se puedan convertir en componentes (por ejemplo, la tarjeta de un registro, la lista de registros y el detalle de una relación) y justifique cada elección. Cree la carpeta src/components y, si lo desea, mueva los datos escritos a mano a un archivo src/datos.js. Documente los tres candidatos y la estructura de carpetas con capturas.

**Respuesta:**

Tres fragmentos de Vet.jsx pasaron a componentes: el insumo (InsumoCard, se repite), la cita (CitaCard, se repite y tiene su relación anidada) y el consumo (ConsumoItem, elemento de la relación N:M). Los datos se movieron a src/datos.js.



```
src/Vet.jsx
-----------
 10    return (
 11      <section className="seccion-vet">
 12        <h1 className="titulo">{aplicacion}</h1>
 13        <p>Consulta base: S/ {precioConsulta.toFixed(2)}</p>
 14        <p>Consulta con IGV ({igv * 100}%): S/ {(precioConsulta * (1 + igv)).toFixed(2)}</p>
 15
 16        <h2 style={{ color: '#1f5f5b' }}>Insumos</h2>
 17        <InsumoLista insumosIniciales={insumos} />
 18
 19        <h2 style={{ color: '#1f5f5b' }}>Citas y sus consumos</h2>
 20        {citas.map((cita) => (
 21          <CitaCard
 22            key={cita.id}
 23            mascota={cita.mascota}
 24            dueno={cita.dueno}
 25            servicio={cita.servicio}
 26            hora={cita.hora}
 27            estado={cita.estado}
 28            veterinario={cita.veterinario.nombre}
 29            consumos={cita.consumos}
 30          />
 31        ))}
 32      </section>
 33    )
 34  }
 35
 36  export default Vet
 37
```

## Ejercicio 2 — Crear el primer componente funcional

Cree en su propio archivo (src/components) un componente funcional con nombre en mayúscula que devuelva el JSX de un registro de su entidad principal y use export default. Impórtelo en App.jsx y muéstrelo dos veces. Explique con sus palabras qué ocurre si el nombre del componente se escribe en minúscula.

**Respuesta:**

InsumoCard es una función con nombre en mayúscula y export default; se importa en la lista y se muestra una vez por insumo. Si el nombre fuera minúscula, React lo trataría como una etiqueta HTML y no como un componente.

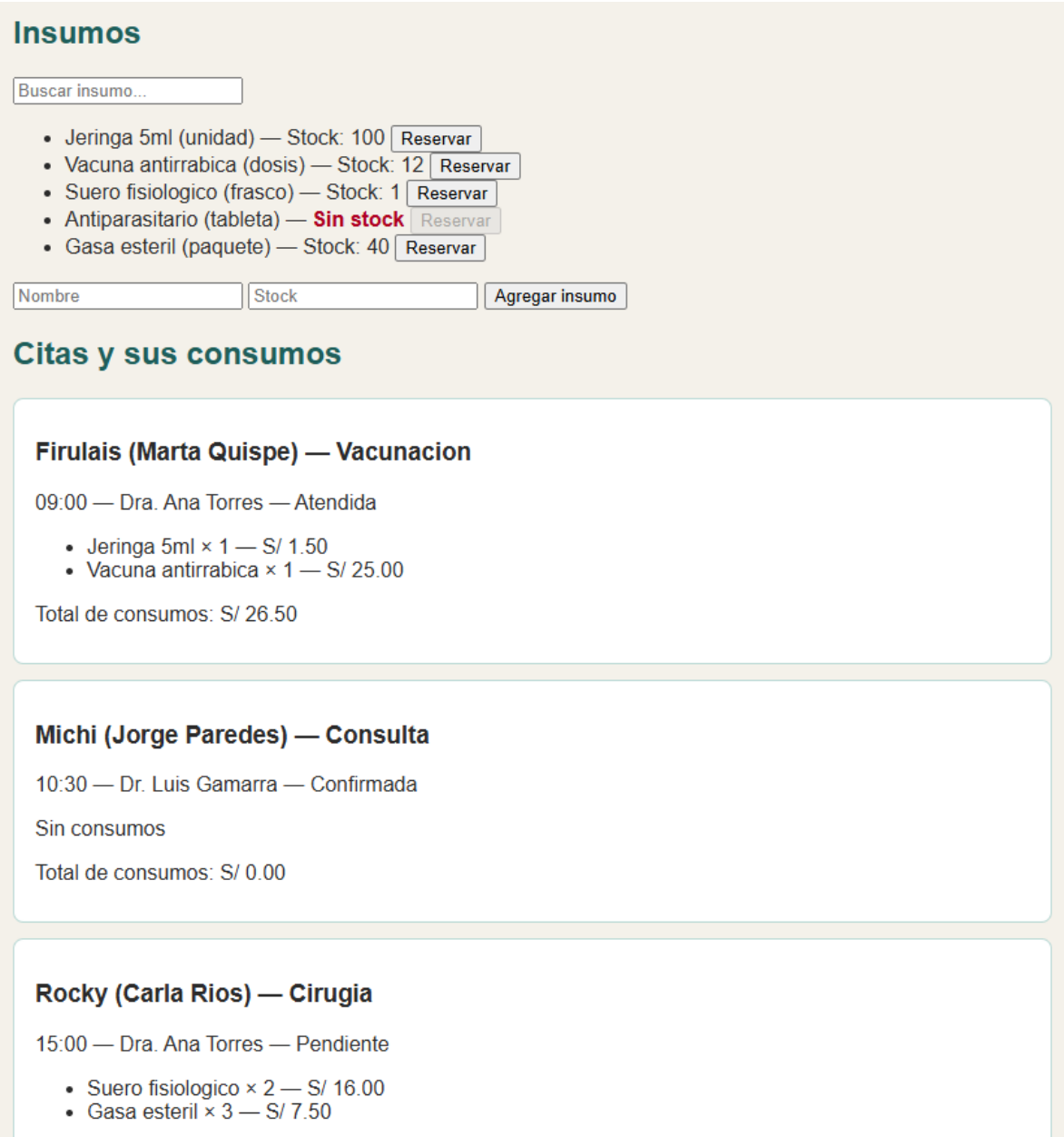
src/components/InsumoCard.jsx

```jsx
import { useState } from 'react'

// Un insumo con un boton de estado local: cada tarjeta lleva su propio contador.
// "unidad" tiene valor por defecto.
function InsumoCard({ nombre, unidad = 'unidad', stock }) {
  const [reservados, setReservados] = useState(0)
  const agotado = stock === 0 || reservados >= stock

  return (
    <li>
      {nombre} ({unidad}) —{' '}
      {stock === 0 ? <span className="sin-stock">Sin stock</span> : `Stock: ${stock}`}{' '}
      <button onClick={() => setReservados(reservados + 1)} disabled={agotado}>
        Reservar
      </button>
      {reservados > 0 && <strong> Reservados: {reservados}</strong>}
    </li>
  )
}

export default InsumoCard
```




## Ejercicio 3 — Pasar datos con props y armar la lista

Modifique el componente para que reciba los datos del registro mediante props desestructuradas (por ejemplo, nombre, precio y stock) y agregue un valor por defecto a una de ellas. Cree un segundo componente que reciba el arreglo de registros y muestre un componente por cada uno con map() y la key basada en el id. Documente el código y el resultado en el navegador.

**Respuesta:**

InsumoCard recibe nombre, unidad y stock desestructurados; unidad tiene valor por defecto ("unidad"). InsumoLista recibe el arreglo y muestra un InsumoCard por cada insumo con map() y key={insumo.id}.

src/components/InsumoLista.jsx

```
45          <ul>
46            {visibles.map((insumo) => (
47              <InsumoCard key={insumo.id} nombre={insumo.nombre} unidad={insumo.unidad} stock={insumo.stock} />
48            ))}
49          </ul>
50        )}
51
```

## Ejercicio 4 — Crear un componente contenedor con children

Cree un componente contenedor (por ejemplo, Tarjeta) que reciba un título y children, y use su estilo para envolver el contenido de al menos un registro o una sección. Documente el código y explique con sus palabras qué equivalente tiene children en los Templates de Django de la Semana 6.

**Respuesta:**

Tarjeta recibe titulo y children y envuelve cada cita. children equivale al {% block %} de la herencia de Templates de Django: el hueco que rellena quien usa el componente.

src/components/Tarjeta.jsx

```jsx
// Contenedor reutilizable: recibe un titulo y children (lo que se escribe entre sus etiquetas).
function Tarjeta({ titulo, children }) {
  return (
    <div className="tarjeta-cita">
      <h3>{titulo}</h3>
      {children}
    </div>
  )
}

export default Tarjeta
```

## Ejercicio 5 — Componentes dentro de componentes: la relación 1:N

Cree un componente para su relación 1:N (por ejemplo, el pedido con sus detalles) que use otro componente para cada elemento anidado y muestre un total calculado con JavaScript. Convierta los nombres de los datos de la API a nombres de props cuando sea necesario. Explique con sus palabras por qué un componente no debe modificar sus props y pruebe qué mensaje aparece si lo intenta.

**Respuesta:**

CitaCard usa un ConsumoItem por cada consumo y calcula el total con reduce(). El dato de la API "dueno" se recibe como "propietario". Una prop no se modifica porque es un dato del padre: cambiarla no redibuja la pantalla ni cambia el dato original. Al intentarlo, ESLint marca el error "Modifying component props or hook arguments is not allowed".

src/components/CitaCard.jsx

```jsx
import Tarjeta from './Tarjeta.jsx'
import ConsumoItem from './ConsumoItem.jsx'

// Relacion 1:N con listas anidadas: la cita usa ConsumoItem por cada consumo.
// El prop de la API "dueno" se recibe como "propietario" (nombre propio del componente).
function CitaCard({ mascota, dueno: propietario, servicio, hora, estado, veterinario, consumos = [] }) {
  const total = consumos.reduce((suma, c) => suma + Number(c.subtotal), 0)

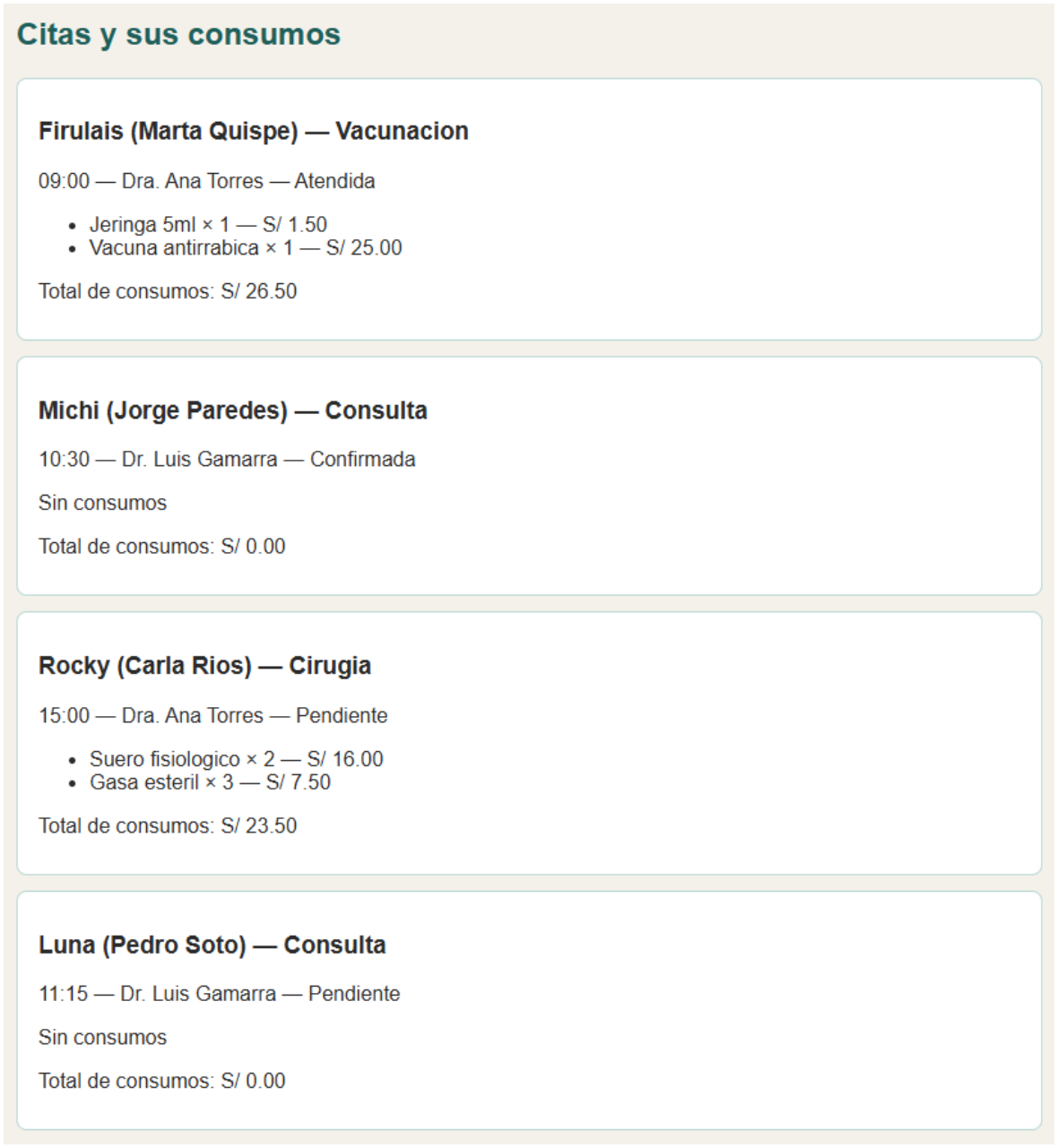
  return (
    <Tarjeta titulo={`${mascota} (${propietario}) — ${servicio}`}>
      <p>
        {hora} — {veterinario} — {estado}
      </p>
      {consumos.length === 0 ? (
        <p>Sin consumos</p>
      ) : (
        <ul>
          {consumos.map((consumo) => (
            <ConsumoItem
              key={consumo.insumo}
              insumo={consumo.insumo}
              cantidad={consumo.cantidad}
              subtotal={consumo.subtotal}
            />
          ))}
        </ul>
      )}
      <p>Total de consumos: S/ {total.toFixed(2)}</p>
    </Tarjeta>
  )
}

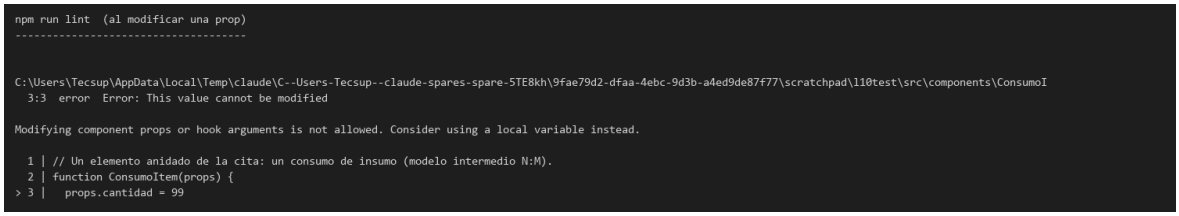
export default CitaCard
```

src/components/ConsumoItem.jsx

```jsx
// Un elemento anidado de la cita: un consumo de insumo (modelo intermedio N:M).
function ConsumoItem({ insumo, cantidad, subtotal }) {
  return (
    <li>
      {insumo} × {cantidad} — S/ {Number(subtotal).toFixed(2)}
    </li>
  )
}

export default ConsumoItem
```




## Ejercicio 6 — Agregar estado y un evento de clic

Agregue al componente del registro un botón que aumente un contador local con useState y muestre el valor solo cuando sea mayor que cero. Desactive el botón cuando corresponda según un campo del registro. Compruebe que cada componente conserva su propio contador. Explique con sus palabras por qué una variable normal (let) no actualiza la pantalla.

**Respuesta:**

Botón Reservar con useState y onClick. El contador solo se ve si es mayor que 0 y el botón se desactiva sin stock o al reservar todo (Suero con stock 1 y Antiparasitario con stock 0). Cada tarjeta tiene su propio

contador. Una variable let no actualiza la pantalla porque React solo redibuja cuando cambia el estado. Antes: sin reservas. Después: Jeringa con 3 clics y Suero con 1.

```
src/components/InsumoCard.jsx
-----------------------------
 1  import { useState } from 'react'
 2
 3  // Un insumo con un boton de estado local: cada tarjeta lleva su propio contador.
 4  // "unidad" tiene valor por defecto.
 5  function InsumoCard({ nombre, unidad = 'unidad', stock }) {
 6    const [reservados, setReservados] = useState(0)
 7    const agotado = stock === 0 || reservados >= stock
 8
 9    return (
10      <li>
11        {nombre} ({unidad}) —{' '}
12        {stock === 0 ? <span className="sin-stock">Sin stock</span> : `Stock: ${stock}`}{' '}
13        <button onClick={() => setReservados(reservados + 1)} disabled={agotado}>
14          Reservar
15        </button>
16        {reservados > 0 && <strong> Reservados: {reservados}</strong>}
17      </li>
18    )
19  }
20
21  export default InsumoCard
```







## Ejercicio 7 — Un campo controlado que filtra la lista

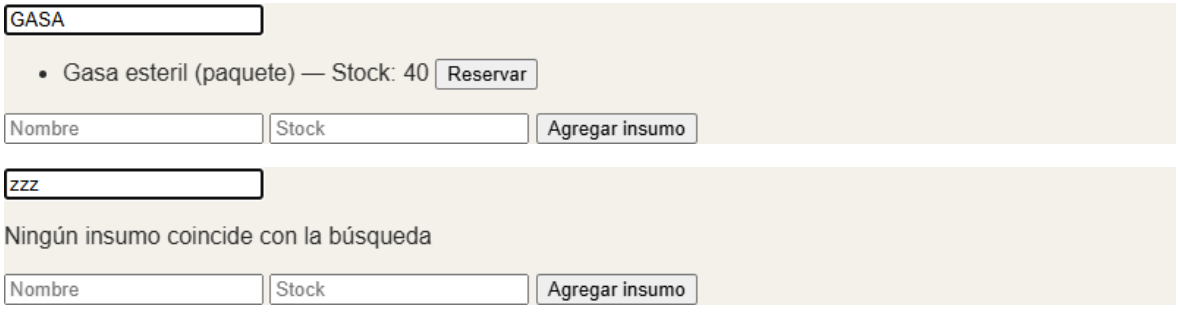
En el componente que contiene la lista, agregue un campo de búsqueda controlado (value y onChange con e.target.value) que filtre los registros por nombre sin distinguir mayúsculas de minúsculas. Muestre un mensaje cuando ningún registro coincida. Documente el código y dos búsquedas con sus resultados.

**Respuesta:**

Campo de búsqueda controlado (value y onChange con e.target.value) que filtra por nombre sin distinguir mayúsculas y muestra un mensaje si nada coincide. Búsquedas: "GASA" da 1 resultado; "zzz" muestra el mensaje.

```
src/components/InsumoLista.jsx
------------------------------
  7    const [insumos, setInsumos] = useState(insumosIniciales)
  8    const [busqueda, setBusqueda] = useState('')
  9    const [nombre, setNombre] = useState('')
 10    const [stock, setStock] = useState('')
 11    const [error, setError] = useState('')
 12
 13    const visibles = insumos.filter((i) =>
 14      i.nombre.toLowerCase().includes(busqueda.toLowerCase()),
 15    )
 16
 17    const enviar = (e) => {
 18      e.preventDefault()
 19      if (nombre.trim() === '') {
 20        setError('El nombre es obligatorio.')
```

```
src/components/InsumoLista.jsx
------------------------------
 38          placeholder="Buscar insumo..."
 39          value={busqueda}
 40          onChange={(e) => setBusqueda(e.target.value)}
 41        />
 42        {visibles.length === 0 ? (
 43          <p>Ningún insumo coincide con la búsqueda</p>
 44        ) : (
```



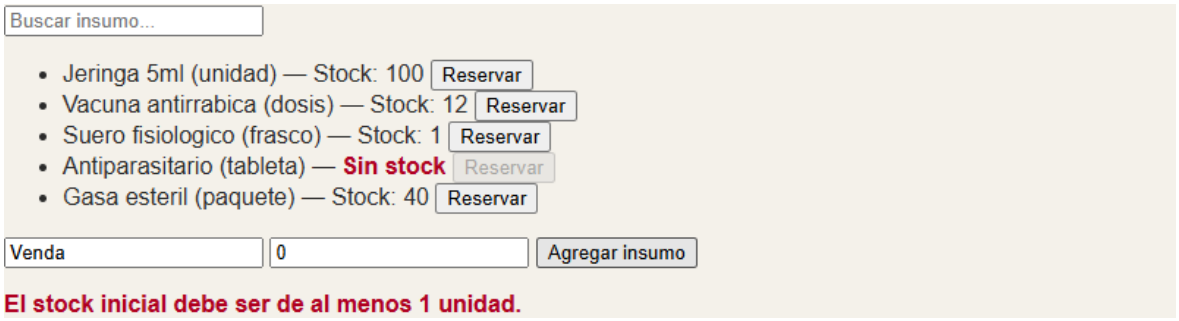

## Ejercicio 8 — Formulario controlado con onSubmit y validación

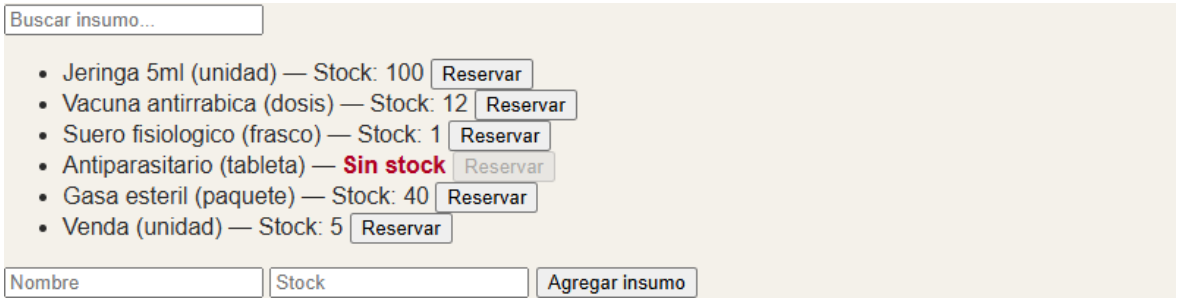
Agregue un formulario controlado con al menos dos campos que agregue un registro nuevo a la lista guardada en el estado, usando onSubmit, preventDefault y un arreglo nuevo (sin push). Valide al menos una regla tomada de su API de la Semana 8 y muestre un mensaje de error cuando no se cumpla. Limpie los campos después de un envío correcto. Ejecute npm run lint y npm run build, documente la salida y explique con sus palabras qué falta para guardar el registro en la base de datos de Django.

**Respuesta:**

Formulario controlado (nombre y stock) con onSubmit y preventDefault. Regla de la API de la Semana 8: el stock inicial debe ser de al menos 1. El registro se agrega con un arreglo nuevo (sin push) y los campos se limpian. Inválido: stock 0 muestra el error. Válido: stock 5 agrega el insumo (la lista pasa de 5 a 6).

```
src/components/InsumoLista.jsx
------------------------------
 22      }
 23      if (Number(stock) < 1) {
 24        setError('El stock inicial debe ser de al menos 1 unidad.')
 25        return
 26      }
 27      const nuevo = { id: Date.now(), nombre: nombre.trim(), unidad: 'unidad', stock: Number(stock) }
 28      setInsumos([...insumos, nuevo])
 29      setNombre('')
 30      setStock('')
 31      setError('')
 32    }
 33
 34    return (
 35      <div>
 36        <input
 37          type="text"
 38          placeholder="Buscar insumo..."
 39          value={busqueda}
 40          onChange={(e) => setBusqueda(e.target.value)}
 41        />
 42        {visibles.length === 0 ? (
 43          <p>Ningún insumo coincide con la búsqueda</p>
```



```
npm run lint  /  npm run build
------------------------------
> frontend@0.0.0 lint
> eslint .

> frontend@0.0.0 build
> vite build

[36mvite v8.3.5 [32mbuilding client environment for production...[36m[39m
transforming...
 24 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.47 kB | gzip:  0.30 kB
dist/assets/index-Ckmuafma.css    0.61 kB | gzip:  0.29 kB
dist/assets/index-BUy5OuXW.js   227.09 kB | gzip: 71.20 kB

[32m built in 90ms[39m
```

```
Consola del navegador (errores y advertencias): sin mensajes
```

Falta para guardar en Django: el insumo nuevo vive solo en la memoria del navegador y se pierde al recargar. Hay que enviarlo con fetch (POST a /api/insumos/), habilitar CORS en Django y manejar la respuesta o los errores de la API.

## PARTE 2 — Componentes, estado y eventos para la investigación propia (registrada en el Admin en la Semana 5)

A partir de aquí trabajará sobre los datos de su investigación propia: las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es reorganizar en componentes la interfaz que construyó en la Semana 9 para su investigación y agregarle estado, eventos y un formulario, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

### Ejercicio 9 — Identificar los componentes de la investigación propia

Complete la tabla de equivalencias con su investigación: indique qué componente mostrará un registro, qué botón con estado agregará, qué componente mostrará su relación anidada y qué formulario con qué regla de validación construirá. Anote también qué regla de su API de la Semana 8 usará para la validación.

#### Checklist antes de continuar — Parte 2

| Criterio | Verificación |
|---|---|
| 1. Existen al menos tres componentes funcionales, cada uno en su archivo dentro de src/components, con nombre en mayúscula. | Estructura de carpetas + código. |
| 2. Los datos llegan a los componentes por props desestructuradas, y hay un valor por defecto o un uso de children. | Código de los componentes. |
| 3. Una lista de componentes se muestra con map() y key en el componente. | Código + captura. |

| | |
|---|---|
| 4. Una relación 1:N se muestra con un componente dentro de otro componente. | Código + captura. |
| 5. Un componente tiene estado con useState y un evento onClick que lo cambia. | Capturas antes y después del clic. |
| 6. Un campo controlado (value y onChange) filtra la lista. | Capturas de dos búsquedas. |
| 7. Un formulario controlado con onSubmit y preventDefault agrega un registro sin modificar el arreglo y valida una regla con mensaje de error. | Capturas del envío válido y del inválido. |
| 8. La consola del navegador no muestra advertencias y npm run lint y npm run build terminan sin errores. | Capturas de la consola y de la terminal. |
| 9. Se explica por escrito qué resuelven las props y el estado, y qué falta para guardar los datos en Django. | Párrafo de justificación. |

### Ejercicio 10 — Componentes y props de la investigación propia

Cree al menos tres componentes funcionales para su investigación (el registro, la lista y un componente contenedor o de relación), cada uno en su archivo, que reciban los datos por props y se muestren con map() y key. Incluya un valor por defecto o el uso de children.

### Ejercicio 11 — Estado, eventos y buscador en la investigación propia

Agregue a un componente un botón con estado y un evento de clic, y un campo de búsqueda controlado que filtre la lista. Documente el antes y el después de cada interacción.

### Ejercicio 12 — Relación anidada y formulario en la investigación propia

Muestre la relación 1:N o el modelo intermedio con un componente dentro de otro, y construya un formulario controlado con onSubmit, preventDefault y validación que agregue un registro a la lista guardada en el estado. Documente un envío válido y uno inválido.

### Ejercicio 13 — Verificar la interfaz de la investigación propia

Verifique que la consola del navegador no muestra advertencias (por ejemplo, de key o de un campo con value sin onChange), ejecute npm run lint y npm run build, y documente la salida. Compruebe que las carpetas node_modules y dist no están en el repositorio.

### Ejercicio 14 — Publicar en GitHub

Actualice el README.md describiendo los componentes creados, qué hace cada uno y cómo ejecutar el frontend (npm install y npm run dev). Haga commit y push del proyecto de React, sin las carpetas node_modules ni dist. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

### Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de los componentes, la página antes y después de cada interacción y la salida de npm run lint y npm run build).
- Repositorio en GitHub actualizado, con la carpeta frontend, el README.md y el código correspondiente, sin node_modules ni dist.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Repositorio: https://github.com/RXDG2908/DesAplEmp-S02

## Conclusiones:

Los componentes dividen la interfaz en piezas reutilizables que reciben datos por props.

useState guarda lo que cambia y React redibuja solo cuando cambia el estado.

Los campos controlados y onSubmit permiten filtrar y agregar registros validando reglas de la API.

Falta conectar con Django mediante fetch para guardar los datos.

| |
|---|
| Dividir la interfaz en componentes deja cada parte en su propio archivo y permite reutilizarla, como un include en los Templates de Django. |
| Las props pasan los datos del componente padre al hijo y son de solo lectura: el hijo no las cambia. |
| children permite armar contenedores que envuelven cualquier contenido, parecido a un bloque en la herencia de Templates. |
| El estado guarda lo que cambia en la pantalla; React solo vuelve a dibujar cuando cambia, por eso una variable normal no sirve. |
| Un campo controlado toma su valor del estado y lo actualiza al escribir, lo que permite filtrar la lista mientras se busca. |
| El formulario controlado valida la misma regla de la API, stock de al menos uno, y agrega el insumo sin modificar la lista original. |
| Lo que falta es guardar en la base de datos: hoy el insumo nuevo se pierde al recargar la página. |